

NxtLight Research
nxtlight.com

Reed Framework: Analytical Modules and Their Functions

A Technical Reference for Network Performance Analysis

Dr. William Reed
NxtLight Research

Document Version 2.2 | May 2026

Aligned with Theoretical Reference Rev 2.2. Includes all seventeen analytical modules, a per-module summary table, an updated hub diagram (Figure 1), and integrated content from the Theoretical Foundations document Rev 1.5.

The Science and Value of Network Analysis

Why Networks Matter

Modern science and engineering have converged on a surprising insight: many of the most important systems in the natural and built world — the internet, power grids, metabolic pathways, social relationships, neural circuits, financial markets, and ecological food webs — share a common organizational principle. They are all networks: collections of interacting components whose collective behavior cannot be understood by studying any single component in isolation. The interdependence is the phenomenon.

Two papers published in 1998 and 1999 transformed the field. Watts and Strogatz (1998) showed that many real networks share a "small-world" property: despite having thousands or millions of nodes, any two nodes are connected by surprisingly short paths, while local clustering remains high. Barabási and Albert (1999) showed that many of the same networks follow a scale-free degree distribution $P(k) \propto k^{-\gamma}$, in which most nodes have few connections but a small number of hubs have enormously many. Network structure directly governs system performance, resilience, and behavior.

The Four Analytical Traditions

The Reed Framework simultaneously applies seventeen analytical modules organized into four intellectual traditions, each asking a different question about the same simulation.

The information-theoretic tradition (blue) asks: how much directed causal information is flowing, which paths are direct versus mediated, how much organizational memory the system has stored, and how informationally isolated its communities are? Modules: Transfer Entropy, Hub Identification, Partial TE/Granger, Entropy Production Rate, Information Entropy Suite, Information Bottleneck.

The dynamical systems tradition (green) asks: is the system stable or chaotic, what equations govern it, does it lie on a low-dimensional attractor, and does it revisit the same regions of state space? Modules: Lyapunov Exponent, SINDy, Diffusion Maps, TARP, Spectral Radius Tracking, Recurrence Quantification Analysis.

The topological data analysis tradition (purple) asks: what is the global shape of the trajectory through state space, independent of any coordinate system? Module: Persistent Homology.

The algebraic and graph-theoretic tradition (gold) asks: how does the network partition, how does information diffuse, how many external control signals are needed to steer it, and does nonlinearity change that count? Modules: Graph Laplacian, Community Detection, Network Controllability (Linear), Nonlinear Controllability.

The Hebbian Plasticity and Echo-State Reservoir Module

The Hebbian Plasticity and Echo-State Reservoir module is the learning mechanism at the core of the NxtLight Framework. Unlike the sixteen analytical modules that observe and measure a running simulation, this module actively drives the co-evolution of network topology and signal. It implements two interacting processes simultaneously: a Hebbian conductance update rule that strengthens well-used edges and prunes underused ones, and an echo-state reservoir dynamic

that governs how the network's recurrent signal memory evolves as topology changes. Together these processes are the reason the framework simulates a genuinely co-evolving network rather than a static graph with signals layered on top.

The Hebbian conductance update follows the rule $\eta_{oij}(t+1) = \eta + Ir \cdot A_i \cdot A_j \cdot (1 - \eta) - D \cdot \eta$, derived directly from Hebb's original postulate (Hebb, 1949) that neurons that fire together wire together. In the NxtLight context, A_i and A_j are the activation levels of the two nodes at the endpoints of edge (i,j) , Ir is the Hebbian learning rate, and D is a decay constant that gradually reduces conductance on edges that carry little traffic. Edges connecting highly co-active nodes accumulate conductance over time and become signal highways; edges connecting rarely co-active nodes decay toward zero and are eventually pruned. The net effect is a topology that continuously reorganizes itself to match the causal structure of the signal it carries.

The echo-state reservoir aspect governs how much temporal memory the network retains. In reservoir computing theory (Jaeger, 2001), a recurrent network stores a fading memory of past inputs in its internal state, and the richness of that memory depends on the spectral radius $\rho(A(t))$ of the adjacency matrix. When $\rho < 1$, the network satisfies the echo state property: past inputs fade exponentially, and the current state is entirely determined by the recent input history. When $\rho = 1$, the network operates at the edge of chaos with maximum memory capacity. When $\rho > 1$, past perturbations grow without bound and the reservoir becomes unstable. Because every Hebbian rewiring event changes the adjacency matrix, it changes ρ as well. The Hebbian update and the reservoir operating point are therefore inseparable: learning moves the network through structural space, and each structural move changes the dynamical regime in which all analytical modules subsequently operate. The Spectral Radius Tracking module (Module 13) monitors $\rho(A(t))$ continuously and reports when the operating point drifts away from the edge-of-chaos target.

The four analytical traditions each observe a different consequence of this underlying learning process. The information-theoretic modules measure how Hebbian learning reshapes causal information flow and hub structure as high-conductance edges become dominant signal pathways. The dynamical systems modules measure how learning shifts the network's stability regime, its attractor geometry, and its equation of motion as the effective dimensionality of the dynamics changes. The topological data analysis module detects how the trajectory through state space changes shape as learning locks in or breaks specific recurrence patterns. The algebraic and graph-theoretic modules track how learning alters the network's spectral properties, community boundaries, and controllability as the Laplacian and adjacency structure evolve. None of the seventeen analytical modules can be fully interpreted without understanding that the topology they are measuring is itself an ongoing product of the Hebbian process described here.

Three Thermodynamic Phases

The framework defines a dimensionless control parameter $\Phi = P/D$, where P is the edge-addition probability per timestep and D is the edge-removal probability per timestep. Three distinct dynamical phases are identified as Φ varies.

In the Crystallized Phase ($\Phi > \Phi_c$), promotion dominates: edges accumulate around a small number of super-hubs, H_{deg} falls toward zero, and dynamics freeze into a near-static configuration with low transfer entropy.

In the Dissipative Phase ($\Phi_d < \Phi < \Phi_c$), promotion and demotion balance: the network maintains a stationary Pareto-like degree distribution with intermediate H_{deg} , active directed information flow (high TE_{out}), and rich dynamics near the edge of chaos. This is the primary target of the analytical modules.

In the Dissolved Phase ($\Phi < \Phi_d$), demotion dominates: the graph sparsifies, communities dissolve, H_{deg} rises toward its maximum, TE_{out} collapses, and the Fiedler value λ_2 approaches zero.

Multi-Method Value: Why All Seventeen Run Simultaneously

The value of running all seventeen modules on the same simulation cannot be overstated. A network that appears stable by Lyapunov analysis may simultaneously be losing community structure (detected by modularity tracking), which is an early warning of fragmentation that the Lyapunov exponent alone cannot detect. A network whose transfer entropy heatmap shows strong directed flow may simultaneously be approaching an uncontrollable configuration (detected by controllability analysis). A network whose persistent homology shows a persistent loop may simultaneously have a small Fiedler value, meaning the periodic behavior is confined to a nearly isolated subgraph. These cross-module insights are the primary scientific output of the framework.

Figure 1. Module Hub Diagram — All Seventeen Analytical Modules

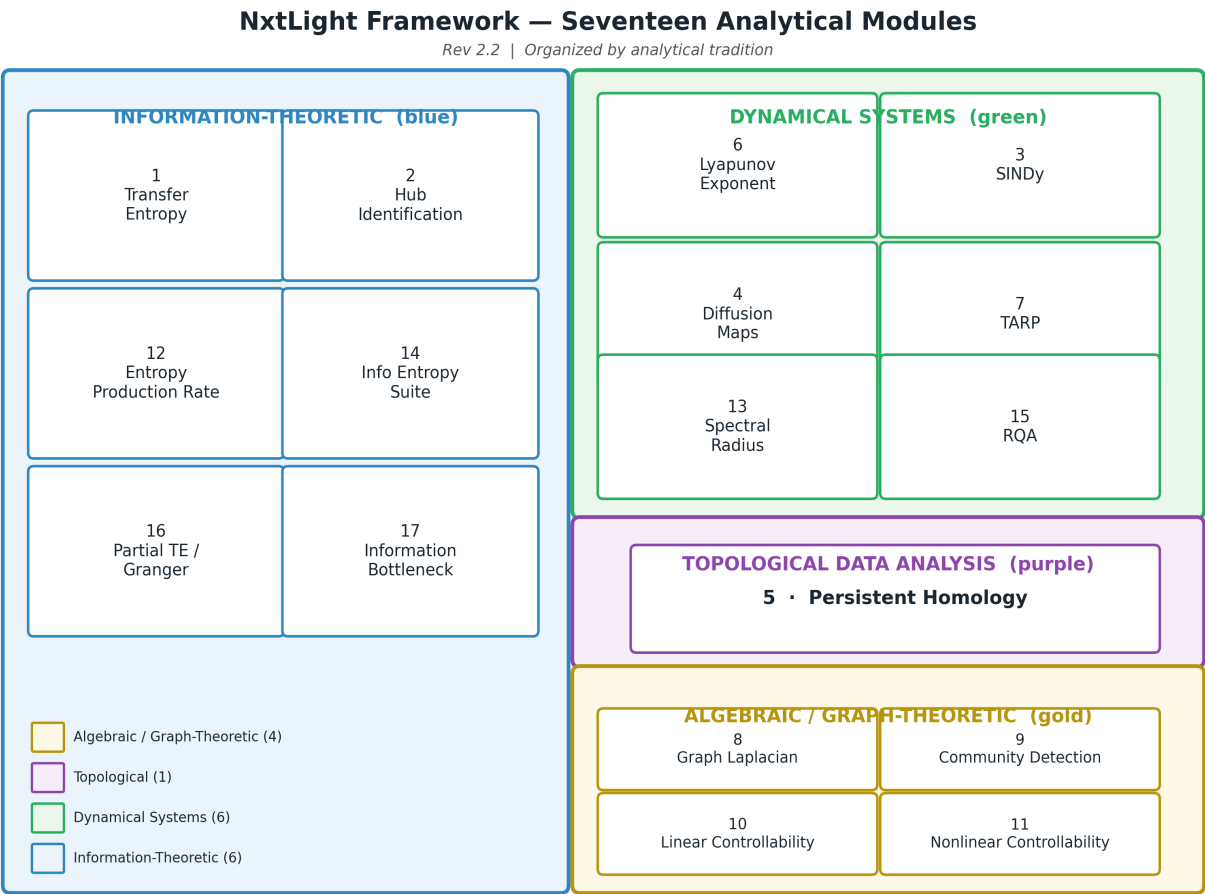


Figure 1. All seventeen analytical modules of the NxtLight Framework Rev 2.2, organized by the four intellectual traditions. Module numbers match the section numbers in this document. The Hebbian Plasticity and Echo-State Reservoir module, described in a separate section above, is the learning mechanism whose continuous rewiring produces the topology that all seventeen modules analyze.

Cross-Module Convergence Dashboard

The convergence dashboard aggregates one representative metric from each of the framework's core analytical modules onto a single time-axis chart. Viewing all metrics simultaneously makes the multi-module signature of each network regime immediately visible, and highlights the moments when metrics from different traditions converge or diverge — which are the most scientifically interesting moments in any simulation.

Module	Dashboard Metric	Units	What to Watch
Hub Identification (§2)	hub_score_top3_mean	dimensionless	Rises in dissipative phase as structural and dynamical hubs emerge; falls in crystallized and dissolved phases
Transfer Entropy (§1)	TE_out (mean)	bits/step	Drops near zero in dissolved phase; hub spikes emerge in dissipative phase
Partial TE / Granger (§16)	PTE_out (mean)	bits/step	Lower than TE_out when indirect paths dominate; rises when direct causal structure sharpens
Lyapunov Exponent (§6)	$\lambda_{\max}$	bits/step	Sign change from negative to positive marks stability transition; near zero = edge of chaos
SINDy (§3)	Fit residual ($1-R^2$)	dimensionless	Low residual = successful equation recovery; high residual in dissolved or chaotic phases
Diffusion Maps (§4)	DC ₁ variance	dimensionless	High variance = active slow manifold; low = dynamics collapsed to fixed point
TARP (§7)	Coverage deviation	dimensionless	Above zero = attractor structure present; near zero = noise-dominated dynamics
Persistent Homology (§5)	β_1 max persistence	normalized ϵ	Persistent loop = periodic or quasi-periodic dynamics; absence = fixed point or chaos
Entropy Production (§12)	σ (normalized)	dimensionless	High σ in dissipative phase; drop signals approach to equilibrium (crystallized) or dissolution
Graph Laplacian (§8)	λ_2 Fiedler value	dimensionless	Near zero = near-disconnection; largest = highly integrated; spectral gap = mixing rate
Community Detection (§9)	Q modularity	dimensionless	Drop signals community merger or fragmentation; joint $Q-\lambda_2$ determines regime
Controllability — Linear (§10)	$n_{D/N}$ driver fraction	dimensionless	Rising = increasing intervention cost; high in hub-dominated (crystallized) phase
Controllability — Nonlinear (§11)	$r = n_{D\{NL\}}/n_{D\{L\}}$	dimensionless	$r > 1$: nonlinearity makes system harder to control than linear predicts
Spectral Radius (§13)	$\rho(A(t))$	dimensionless	Near 1 = edge-of-chaos operating point; $>> 1$ = unstable reservoir; $<< 1$ = over-damped
Info Entropy Suite (§14)	H_deg	bits	Structural order parameter; falls as hubs form; rises as network dissolves

RQA (§15)	LAM (laminarity)	dimensionless	Spike precedes phase transition; DET measures determinism of state-space revisitation
Info Bottleneck (§17)	IB efficiency (mean)	dimensionless	Near 0 = communities informationally isolated (echo chambers); near 1 = high inter-community coupling
Entropy Production (§12)	d _i S/dt, d _e S/dt	dimensionless	Split reports internal production vs. export; verifies far-from-equilibrium operation

Unified Equations Reference

§	Equation	Origin	Interpretation
§3	$ds/dt = d_{iS}/dt + d_{eS}/dt$	Prigogine (1984)	Dissipative structure entropy decomposition
§4	$C = B \log_2(1 + \text{SNR})$	Shannon (1948)	Edge channel capacity
§5	$G_{\{t+1\}} = \text{Rewire}(G_t, A_t, \theta_P, \theta_D)$	Reed Framework	Topology update rule
§5	$A_{\{t+1\}} = f(A_t, G_t, \eta_t, u_t)$	Reed Framework	Node dynamics update
§1	$TE(X \rightarrow Y) = H(Y_{\{t+1\}} Y_t^k) - H(Y_{\{t+1\}} Y_t^k, X_t^l)$	Schreiber (2000)	Directed causal information flow
§2	$\zeta_i = \sum_{j \neq i} TE(i \rightarrow j)$	Reed Framework	Causal out-degree (hub score)
§6	$\ \delta x(t)\ \approx e^{\lambda_{\max} t} \ \delta x(0)\ $	Oseledec / Rosenstein	Stability classification
§12	$\eta_{\{ij\}}(t+1) = \eta + lr \cdot A_i \cdot A_j \cdot (1-\eta) - D \cdot \eta$	Hebb (1949)	Hebbian conductance update
§8	$L = D - A; \lambda_2(L) = \text{Fiedler value}$	Fiedler (1973)	Algebraic connectivity
§9	$Q = (1/2m) \sum_{\{ij\}} [A_{\{ij\}} - k_i k_j / 2m] \delta(c_i, c_j)$	Newman (2006)	Modularity (community strength)
§10	$N_D = n - \text{rank}(A)$	Yuan et al. (2013)	Minimum driver node count (linear)
§10	$W_c = \int_0^T e^{A t} B B^T e^{A^T t} dt$	Kalman (1963)	Controllability Gramian
§11	$r = n_D^{\{NL\}} / n_D^{\{L\}}$	Hermann & Krener (1977)	Nonlinear controllability ratio
§13	$\rho(A) = \max \lambda_i(A) $	Koopman / Mezić	Spectral radius; edge-of-chaos at $\rho=1$
§12	$\sigma = (1/T) KL[P[Y] P[\tilde{Y}]]$	Seifert (2012)	Entropy production rate

1. Transfer Entropy Estimation

Module	Function	Key Inputs	Key Outputs
1 Transfer Entropy Estimation	Measures directed statistical dependence between node time series; identifies which nodes causally drive others beyond correlational co-movement.	Node activation histories $A_i(t)$, sliding window W (default 200 steps), embedding dimensions k, l	$N \times N$ TE matrix (bits/step); $TE_out(i)$ per node; causal hub ranking; Gini coefficient of TE_out

Tradition: Information-Theoretic

Transfer entropy (TE) is a model-free, information-theoretic measure that quantifies directed statistical dependence between two time series. Introduced by Schreiber (2000), it measures how much knowing the past state of a source node reduces uncertainty about the future state of a target node, beyond what the target's own history already explains.

$$TE(X \rightarrow Y) = H(Y_{t+1} \mid Y_t^{(k)}) - H(Y_{t+1} \mid Y_t^{(k)}, X_t^{(l)})$$

TE is asymmetric by construction — $TE(X \rightarrow Y)$ need not equal $TE(Y \rightarrow X)$ — making it the natural measure of directed causality in a network. In the Reed Framework, TE is estimated between every ordered pair of nodes using the Kraskov–Stögbauer–Grassberger (KSG) k -nearest-neighbor estimator [3], which avoids the binning artifacts of histogram-based estimators.

The full pairwise TE matrix is computed at user-defined intervals and displayed as a heatmap in the GUI. Rows with high average outgoing TE identify broadcaster or driver nodes; columns with high incoming TE identify integrator or hub nodes. The causal out-degree $\zeta_i = \sum_{j \neq i} TE(i \rightarrow j)$ identifies functional driver nodes and serves as the primary phase diagnostic: high in the dissipative phase, low in both crystallized and dissolved phases.

Example Application: Epidemic Spreading

In a susceptible-infected-recovered (SIR) epidemic model on a scale-free contact network, the TE module identifies primary spreaders — nodes with the highest $TE(i \rightarrow j)$ summed over all j — even before any individual has recovered. This causal identification outperforms simple degree ranking because it accounts for temporal structure: a high-degree node infected late contributes less TE than a moderate-degree node that seeds many early infections.

2. Hub Identification

Module	Function	Key Inputs	Key Outputs
2 Hub Identification	Ranks every node on a composite score combining structural position, neighborhood diversity, and causal influence; identifies functional hubs that are genuinely critical.	Betweenness centrality BC_i; degree entropy DE_i; TE_out(i) or PTE_out(i) from Module 1 or 16	Hub score H_i for all nodes; ranked top-K hub list (default K=3); hub_score_top3_mean dashboard metric

Tradition: Information-Theoretic

Hub identification uses a composite scoring system combining three independent signals — betweenness centrality, degree entropy, and outgoing transfer entropy — into a single ranked hub score. Any single metric can mislead: a high-degree node may be peripheral in information routing, while a moderately connected node at a structural bottleneck may be the true network driver.

Betweenness centrality (Freeman, 1977) counts the fraction of all shortest paths that pass through a given node. Degree entropy captures the heterogeneity of a node's neighborhood — high degree entropy implies the node bridges structurally diverse parts of the network, a hallmark of a genuine hub in the Barabási–Albert sense [11]. Outgoing transfer entropy provides the dynamical signal: a node that consistently drives the future states of many downstream partners is a hub in a functional rather than purely structural sense.

The composite score is $H_i = w_{bc} \cdot BC_i + w_{de} \cdot DE_i + w_{te} \cdot TE_i$ (default weights: $w_{bc} = 0.40$, $w_{de} = 0.30$, $w_{te} = 0.30$). The three scores are normalized to $[0, 1]$ before weighting. When the Partial TE module (Module 16) is enabled, PTE_out(i) replaces TE_out(i) in the composite score, separating direct causal influence from mediated influence.

Example Application: Power Grid Vulnerability

In a power transmission network simulation, hub identification applied to real and reactive power flow time series reveals substations that rank in the top five by all three metrics simultaneously — high betweenness, high neighborhood entropy, and high outgoing TE. These substations are the most likely cascade failure triggers if overloaded, providing a principled data-driven complement to engineering load-flow analysis.

3. SINDy — Sparse Identification of Network Dynamics

Module	Function	Key Inputs	Key Outputs
3 SINDy — Sparse Identification of Network Dynamics	Recovers interpretable governing equations from simulation data; identifies the smallest set of mathematical terms that accurately describe collective network dynamics.	Aggregate node-state time series $X(t)$; feature library $\Theta(X)$ (polynomial, trigonometric); STLSQ threshold	Sparse coefficient matrix Ξ ; fit residual ($1-R^2$); identified equation terms; surrogate model for parameter sweeps

Tradition: Dynamical Systems

The Sparse Identification of Nonlinear Dynamics (SINDy) algorithm (Brunton, Proctor & Kutz, 2016) recovers equations of motion from time series data by constraining candidate equations to be sparse linear combinations of terms from a user-specified library of candidate functions. The framework approximates $\dot{X} \approx \Theta(X)\xi$, where $\Theta(X)$ is the feature library matrix and ξ is a sparse coefficient vector identified by Sequentially Thresholded Least Squares or LASSO regularization.

SINDy is implemented via the PySINDy library [5]. At each analysis run, the module constructs a polynomial library up to a user-specified order, fits the sparse model, and reports both identified equation coefficients and cross-validation error. In the dissipative phase, SINDy typically recovers a sparse ODE system with 3–7 active terms, confirming that the high-dimensional simulation collapses onto a low-dimensional attractor. In the dissolved phase, no sparse representation exists — which is itself informative about the current regime.

The identified equations serve dual purposes: providing human-interpretable insight into the network's collective behavior, and enabling fast forward prediction through surrogate modeling of the fitted equations.

Example Application: Opinion Dynamics

In a bounded-confidence opinion dynamics simulation, applying SINDy to the time series of mean opinion, variance, and clustering coefficient recovers a two-term model — a linear decay term and a quadratic self-interaction term — that matches the observed trajectory with under 2% error. This immediately reveals logistic-like dynamics, enabling analytical computation of time-to-consensus as a function of connectivity threshold.

4. Slow-Manifold Extraction via Diffusion Maps

Module	Function	Key Inputs	Key Outputs
4 Slow-Manifold Extraction via Diffusion Maps	Reveals the intrinsic low-dimensional geometry of the state-space trajectory using kernel-based nonlinear dimensionality reduction; identifies effective degrees of freedom.	Sequence of joint state vectors $[A_1(t), \dots, A_N(t), w_{\{ij\}}(t), \dots]$; kernel bandwidth ϵ ; sliding window	Diffusion coordinates DC_1, DC_2; manifold dimensionality (spectral gap); 3D trajectory visualization

Tradition: Dynamical Systems

Diffusion maps (Coifman & Lafon, 2006) are a kernel-based nonlinear dimensionality reduction technique that uncovers the intrinsic geometric structure of high-dimensional data. A Gaussian kernel $K(x_i, x_j) = \exp(-\|x_i - x_j\|^2/\epsilon^2)$ defines transition probabilities for a random walk on the data; the eigenvectors of the resulting Markov transition matrix — the diffusion coordinates — provide intrinsic coordinates on the underlying manifold.

The Reed Framework applies diffusion maps to the sequence of network state vectors from a simulation run. A ring-shaped embedding indicates periodic dynamics; a filled cloud indicates higher-dimensional or chaotic behavior; a curve indicates slow relaxation. The effective manifold dimensionality — estimated from the spectral gap between successive eigenvalues — is reported as a diagnostic metric and logged to the XLSX output. In the dissipative phase, SINDy typically finds 3–7 governing terms while diffusion maps confirm a 2–3 dimensional slow manifold; convergence of these two measures is strong evidence for genuine low-dimensional attractor structure.

Example Application: Kuramoto Oscillators

A network of 50 Kuramoto phase oscillators in the partially synchronized phase has high-dimensional individual dynamics, but applying diffusion maps to the full phase-vector time series recovers a near-perfect two-dimensional ring in diffusion coordinates. Despite 50 node variables, the dynamics are effectively governed by two slow degrees of freedom — the amplitude and phase of the collective mode.

5. Persistent Homology

Module	Function	Key Inputs	Key Outputs
5 Persistent Homology	Characterizes the global topological shape of the simulation's trajectory through state space; identifies loops, voids, and components that persist across spatial scales.	Lag-embedded activation state vectors; filtration radius range; Ripser library	Betti numbers β_0 , β_1 , β_2 vs. filtration radius; persistence diagram; β_1 max persistence (dashboard)

Tradition: Topological Data Analysis

Persistent homology (Edelsbrunner, Letscher & Zomorodian, 2002) characterizes the topological shape of data sets by building a sequence of nested simplicial complexes (a filtration) from the data and tracking how topological features — connected components, loops, and voids — appear and disappear as a scale parameter grows. Features that persist across a wide range of scales are topologically significant; short-lived features are treated as noise.

The Reed Framework implements persistent homology via the Ripser algorithm applied to the state-space trajectory — the sequence of high-dimensional state vectors rather than the network topology itself. Betti numbers β_0 (components), β_1 (loops), and β_2 (voids) are tracked as functions of filtration radius and plotted in the GUI. A $\beta_1 = 1$ loop with persistence > 0.3 is the signature of a periodic or quasi-periodic attractor; $\beta_1 = 0$ with all persistence < 0.05 signals noise or a fixed-point attractor.

Persistent homology is particularly powerful in combination with Lyapunov and TARP: if all three agree on periodic or quasi-periodic structure, the identification is highly robust. Persistent homology is the only module that characterizes the global shape of the trajectory independent of any coordinate system or distance metric.

Example Application: Financial Regime Detection

Applied to rolling windows of asset-correlation-matrix time series, persistent homology reveals a sharp increase in H_1 persistence in the weeks before a market crash — the trajectory develops a detectable loop structure as the market enters a correlated stress regime. This homological early-warning signal is distinct from and complementary to traditional volatility measures.

6. Lyapunov Exponent Analysis

Module	Function	Key Inputs	Key Outputs
6 Lyapunov Exponent Analysis	Measures average exponential rate of trajectory divergence; classifies the system as stable, edge-of-chaos, or chaotic; provides the earliest warning of impending phase transitions.	Node-state time series; Rosenstein algorithm; time-delay embedding (Takens); sliding window	λ_{max} (bits/step); sign-change event-bus alert; stability classification (stable/edge/chaotic)

Tradition: Dynamical Systems

The maximal Lyapunov exponent λ_{max} characterizes a system's sensitivity to initial conditions. If $\lambda_{\text{max}} > 0$, nearby trajectories diverge exponentially (chaos); if $\lambda_{\text{max}} < 0$, they converge (stable); if $\lambda_{\text{max}} = 0$, the system is at the edge of chaos. The Reed Framework estimates λ_{max} from node-state time series using the Rosenstein algorithm (1993) in a time-delay embedding following Takens' theorem (1981).

The framework targets the dissipative phase near the edge of chaos, where $\lambda_{\text{max}} \approx 0$. A crossing of λ_{max} through zero from below triggers an event-bus alert — one of the three primary phase-transition signatures (alongside Q dropping below threshold and λ_2 approaching zero). In the opinion polarization case study, λ_{max} crosses zero approximately 30 timesteps before the Fiedler value drops below 0.1, demonstrating that Lyapunov analysis provides an earlier warning of fragmentation than topological methods alone.

Example Application: BGP Routing Security

In a simulated autonomous-system network running a simplified BGP, normal routing table convergence yields $\lambda_{\text{max}} < 0$. When a malicious node injects inconsistent route advertisements, λ_{max} increases toward zero and crosses positive within a few dozen simulation steps — well before routing loops become visible in aggregate traffic data — providing an early dynamical warning of a security breach.

7. TARP — Temporal Attractor Validation

Module	Function	Key Inputs	Key Outputs
7 TARP — Temporal Attractor Validation	Tests whether the simulation trajectory has genuine temporal structure distinguishable from noise; rules out the null hypothesis of purely stochastic dynamics.	Activation time series; 1000 phase-shuffled surrogates; autocorrelation function; Kolmogorov–Smirnov statistic	TARP deviation score; coverage deviation (dashboard); binary: attractor structure present / absent

Tradition: Dynamical Systems

TARP (Tests of Accuracy with Random Points), adapted from Lemos et al. (2023), tests whether the simulated trajectory possesses genuine temporal structure distinguishable from noise. For a given time series, the module computes reference statistics (autocorrelation function, power spectral density, average recurrence time), generates 1000 surrogate time series by random shuffling, and computes a TARP deviation score as the Kolmogorov–Smirnov distance between the original ACF and the mean surrogate ACF.

A large TARP score confirms genuine temporal structure; a near-zero score suggests noise-dominated dynamics. TARP serves as a sanity check: if SINDy recovers a non-trivial equation and λ_{max} is positive but TARP returns near-zero, the result is likely an artifact of insufficient simulation length or excessive measurement noise. High TARP scores combined with persistent H_1 features provide strong converging evidence for a genuine chaotic or quasi-periodic attractor.

Example Application: Echo-State Reservoir Tuning

Scanning the reservoir spectral radius while tracking both λ_{max} and the TARP score simultaneously reveals the edge-of-chaos operating point: the TARP deviation score peaks where $\lambda_{\text{max}} \approx 0$, confirming genuine temporal structure at the computationally optimal regime. Both deep in the stable regime and deep in chaos, the TARP score drops.

8. Graph Laplacian Spectral Analysis

Module	Function	Key Inputs	Key Outputs
8 Graph Laplacian Spectral Analysis	Encodes network topology as an algebraic object; computes the Fiedler value as the primary early-warning indicator of disconnection; reveals diffusion structure and natural graph bisection.	Adjacency matrix A(t); degree matrix D; SciPy sparse eigensolver; normalized Laplacian option	Fiedler value λ_2 ; spectral gap Δ ; Fiedler vector ϕ_2 (node color map); configurable threshold alert option

Tradition: Algebraic and Graph-Theoretic

The graph Laplacian $L = D - A$, where D is the diagonal degree matrix and A is the adjacency matrix, is one of the most fundamental algebraic objects associated with a network. For an undirected graph, L is symmetric positive semi-definite with eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$. The Fiedler value λ_2 is the most diagnostically important: $\lambda_2 = 0$ if and only if the graph is disconnected; larger λ_2 implies faster mixing, higher robustness to node removal, and more tightly integrated overall structure.

The spectral gap ($\lambda_3 - \lambda_2$) determines random-walk mixing time $\tau_{\text{mix}} \approx 1/\Delta$. The largest eigenvalue $\lambda_{\text{max}}(L)$ bounds the rate of epidemic spreading: a pathogen with per-contact transmission rate below $1/\lambda_{\text{max}}(L)$ cannot sustain an epidemic (the epidemic threshold theorem). The Fiedler vector ϕ_2 assigns a real value to each node such that sorting by ϕ_2 and cutting between negative and positive values yields the minimum-bisection partition.

The Reed Framework computes the full Laplacian spectrum at each analysis interval using SciPy's sparse eigensolver. A threshold alert fires when λ_2 drops below a configurable fraction of its initial value. The normalized Laplacian $L_{\text{sym}} = D^{-1/2} L D^{-1/2}$ is available for networks with heterogeneous degree distributions.

Example Application: Detecting Fragmentation in a Communication Network

In a mobile ad hoc network (MANET), monitoring the Fiedler value λ_2 in real time reveals impending disconnection: λ_2 drops sharply as a cluster of nodes moves out of radio range even while the network remains formally connected through a single relay node. The spectral early-warning system detects impending fragmentation while the network is still nominally intact.

9. Community Detection and Modularity Tracking

Module	Function	Key Inputs	Key Outputs
9 Community Detection and Modularity Tracking	Identifies mesoscale community structure; tracks formation, evolution, and dissolution of communities over time; distinguishes adaptive specialization from fragmentation via joint Q - λ_2 analysis.	Current graph $G(t)$; Leiden (default) or Louvain algorithm; modularity Q computation; Jaccard identity tracking	Community assignments c_i ; modularity Q ; number of communities K ; node switches; Q threshold alerts

Tradition: Algebraic and Graph-Theoretic

Most real-world networks are organized into mesoscale communities — subsets of nodes that interact more densely with each other than with the rest of the network. Community structure plays a critical role in network performance: communities can compartmentalize failure, accelerate local consensus, support functional specialization, and create structural vulnerabilities at inter-community boundaries.

The modularity function $Q = (1/2m) \sum_{\{ij\}} [A_{\{ij\}} - k_{ik_j}/2m] \delta(c_i, c_j)$ measures the fraction of edges within communities minus the fraction expected in a random graph with the same degree sequence. Q ranges from 0 (no community structure better than random) to approaching 1 (extremely strong community structure). Values of $Q > 0.3$ are generally considered indicative of significant community structure.

The Reed Framework uses the Leiden algorithm (Traag, Waltman & van Eck, 2019) by default, which guarantees well-connected communities; Louvain [24] is available as a fallback. The temporal evolution of Q is interpreted jointly with λ_2 : rising Q combined with falling λ_2 indicates fragmentation (communities strengthening but losing inter-community connectivity), while rising Q combined with stable or rising λ_2 indicates adaptive specialization.

Example Application: Polarization in Social Network Dynamics

In an opinion dynamics simulation with homophily, the community detection module reveals the onset of polarization long before it would be visible in aggregate statistics. As agents preferentially rewire toward like-minded peers, Q increases monotonically while the number of communities drops from eight to two — the network is consolidating into two opposing camps. λ_2 also declines, confirming that inter-community connectivity is weakening.

10. Network Controllability Analysis (Linear)

Module	Function	Key Inputs	Key Outputs
10 Network Controllability Analysis (Linear)	Identifies the minimum set of driver nodes required to steer the network from any initial state to any target state; tracks how structural evolution changes external intervention requirements.	Directed adjacency matrix $A(t)$; maximum matching algorithm; controllability Gramian W_c ; path-length proxy ($N > 80$)	Driver node set; N_D/N driver fraction; Gramian energy metric; driver node GUI highlighting

Tradition: Algebraic and Graph-Theoretic

Network controllability analysis asks which nodes must be driven externally to steer the network from any initial state to any desired target state in finite time. For a linear system $\dot{x} = Ax + Bu$, the Kalman rank condition requires that the controllability matrix $[B, AB, A^2B, \dots, A^{n-1}B]$ has rank n . Liu, Slotine, and Barabási (2011) [26] translated this into a graph-theoretic result: the minimum number of driver nodes N_D equals n minus the size of the maximum matching of the directed graph.

A counter-intuitive key finding of Liu et al. is that scale-free networks require more driver nodes than Erdős–Rényi random networks of the same size. Hub dominance creates structural dependencies: peripheral nodes' states are determined almost entirely by the hub, leaving them uncontrollable without direct hub access. As the rewiring engine builds the crystallized phase, N_D/N initially falls (emerging hubs give indirect control over many nodes) but then rises as hub-dependency chains deepen.

The controllability Gramian $W_c = \int_0^T e^{At} B B^T e^{A^T t} dt$ encodes the energy cost of reaching each direction in state space. The Reed Framework logs the trace of W_c^{-1} (average minimum energy) and the minimum eigenvalue (hardest-to-reach direction). For $N > 80$, an average-path-length proxy is substituted for computational tractability.

Example Application: Targeted Intervention in Epidemic Networks

In a simulated contact network for epidemic control, the controllability module identifies the minimum set of individuals whose quarantine or vaccination would give public health authorities theoretical ability to steer the epidemic trajectory to elimination. This set evolves as the epidemic progresses — early in the outbreak the driver set is concentrated in high-betweenness nodes; as the epidemic saturates hubs, the driver set expands into previously peripheral nodes.

11. Nonlinear Controllability Analysis

Module	Function	Key Inputs	Key Outputs
11 Nonlinear Controllability Analysis	Applies rigorous Lie-algebra and Koopman-operator theory to determine driver node counts for the explicitly nonlinear NxtLight engine; computes the ratio r showing where linear approximation under- or over-estimates controllability.	Nonlinear vector fields $f(x)$, $g(x)u$; Lie bracket computation; EDMD Koopman approximation from trajectory data	Lie-bracket accessibility rank; Koopman driver count; ratio $r = n_D^{NL}/n_D^L$; divergence alerts

Tradition: Algebraic and Graph-Theoretic

The standard Network Controllability module (Module 10) assumes the network obeys linear equations — a necessary simplification that can misrepresent systems whose dynamics are explicitly nonlinear. The NxtLight engine is nonlinear by design: the Hebbian learning rule introduces a quadratic product $A_i \cdot A_j$, the rewiring thresholds impose bang-bang discontinuities, and the output feedback saturates through a sigmoid.

For a nonlinear system $\dot{x} = f(x) + g(x)u$, accessibility and controllability are determined by the Lie algebra generated by the vector fields f and the control directions g_i (Hermann & Krener, 1977). The Lie bracket $[f, g_i] = \partial g_i / \partial x \cdot f - \partial f / \partial x \cdot g_i$ measures the net motion achievable by alternating between f and g_i . A system is locally accessible at x if the Lie algebra span is full rank at x .

An alternative approach uses Koopman operator theory (Koopman, 1931; Mezić, 2005) to lift the nonlinear system into an infinite-dimensional linear system where standard linear controllability tools apply exactly. The EDMD (Extended Dynamic Mode Decomposition) algorithm (Williams et al., 2015) computes a finite-dimensional Koopman approximation from simulation trajectory data. The module reports both Lie-bracket accessibility rank and Koopman-based minimum driver count, and computes $r = n_D^{NL}/n_D^L$. Values of $r > 1$ indicate that nonlinearity makes the system harder to control than the linear approximation suggests; $r < 1$ indicates additional controllability from nonlinear structure.

12. Entropy Production Rate

Module	Function	Key Inputs	Key Outputs
12 Entropy Production Rate	Directly computes the Prigogine entropy decomposition from simulation data; measures how far from thermodynamic equilibrium the network is at each analysis window.	Activation time series; TUR-based proxy (Vu & Hasegawa, 2020); forward and time-reversed path distributions	$\sigma(t)$ normalized entropy production rate; d_iS/dt internal production; d_eS/dt export component

Tradition: Information-Theoretic

The framework's thermodynamic framing (Prigogine, 1967) introduces entropy decomposition as a foundational concept. The Entropy Production Rate module closes the gap between theory and measurement: by measuring the asymmetry between the forward trajectory and its time-reversal, it quantifies how far from equilibrium the network actually is at each analysis window.

In stochastic thermodynamics (Seifert, 2012; Kawai et al., 2007), the entropy production rate σ in a Markov process is given by the Kullback-Leibler divergence between the path distribution of the forward process and its time-reversal: $\sigma = \lim_{T \rightarrow \infty} \{ (1/T) \text{KL}[P[\gamma] || P[\tilde{\gamma}]] \}$. A system in equilibrium has $\sigma = 0$; a system far from equilibrium has $\sigma \gg 0$.

The module uses the TUR-based (Thermodynamic Uncertainty Relation) proxy of Vu and Hasegawa (2020), which lower-bounds σ from the mean and variance of observable currents — specifically, the net flow of activation across directed edges over the analysis window. The module reports $\sigma(t)$ normalized to $[0, 1]$; d_iS/dt estimated as the divergence of the activation current field; and $d_eS/dt = dS_{\text{total}}/dt - d_iS/dt$. A sudden drop in σ is a leading indicator of crystallization or dissolution.

13. Spectral Radius Tracking

Module	Function	Key Inputs	Key Outputs
13 Spectral Radius Tracking	Computes the spectral radius $\rho(A(t))$ of the adjacency matrix at every analysis window; monitors the edge-of-chaos operating condition for the echo-state reservoir dynamics.	Adjacency matrix $A(t)$; NumPy eigvals ($N < 500$) or SciPy eigs power iteration ($N \geq 500$)	$\rho(A(t))$ time series; edge-of-chaos flag $ \rho - 1 < 0.05$; unstable reservoir alert $\rho \gg 1$

Tradition: Dynamical Systems

Module 12 establishes that the spectral radius $\rho(A(t))$ governs the edge-of-chaos condition for reservoir dynamics and changes with every rewiring event. Spectral Radius Tracking operationalizes this by computing and reporting $\rho(A(t))$ at every analysis window, converting reservoir computing theory directly into an empirical dashboard metric.

The spectral radius is the largest absolute eigenvalue of the adjacency matrix $A(t)$. For the recurrent network: $\rho < 1$ ensures the echo state property (input perturbations fade, reservoir state is determined by input history); $\rho = 1$ is the edge-of-chaos operating point with maximal memory capacity; $\rho > 1$ indicates an unstable reservoir where perturbations grow without bound.

For dense graphs ($N < 500$), the full eigenvalue decomposition is computed using NumPy's eigvals. For sparse large graphs ($N \geq 500$), the power iteration method (or SciPy's eigs with $k=1$) extracts only the dominant eigenvalue, reducing the $O(N^3)$ decomposition to $O(N \cdot T_{\text{iter}})$ where T_{iter} is typically 20–50 iterations. The joint trajectory of $\rho(t)$ and λ_{max} (Lyapunov exponent, Module 6) provides the sharpest available characterization of the network's stability regime at each analysis window.

14. Information Entropy Suite

Module	Function	Key Inputs	Key Outputs
14 Information Entropy Suite	Tracks four simultaneous entropy channels measuring structural disorder, dynamical diversity, temporal complexity, and excess organizational memory; provides multi-band diagnostic coverage of network behavioral regime.	Degree distribution $P(k)$; node-state distribution; aggregate activation time series; excess entropy estimator	H_{deg} ; H_{state} ; H_{samp} (sample entropy); E (excess entropy); EntropyAlerts regime flags

Tradition: Information-Theoretic

Shannon entropy $H(P) = -\sum p_i \log_2(p_i)$ quantifies average uncertainty in a probability distribution. In the Reed Framework it serves as a global multi-band diagnostic tracked in four simultaneous channels: (1) H_{deg} , the Shannon entropy of the degree distribution; (2) H_{state} , the Shannon entropy of the node-state distribution; (3) H_{samp} , the sample entropy of the aggregate node-state time series; and (4) E , the mutual information rate or excess entropy measuring organizational memory accumulated from the network's signal history.

H_{deg} is the most sensitive indicator of structural change. A rewiring event that concentrates connectivity in hub nodes decreases H_{deg} ; one that redistributes connections increases it. Monitoring the rolling variance of H_{deg} enables detection of structural phase transitions — the variance spikes near the critical point even before the mean changes significantly, a signature of critical slowing down. H_{state} captures instantaneous dynamical diversity: synchrony drives H_{state} toward zero; heterogeneous dynamics push it toward its maximum.

The most powerful use of the entropy suite is in combination with the algebraic and graph-theoretic modules. Falling H_{deg} + falling λ_2 + rising Q simultaneously indicates polarization or fragmentation. Rising H_{deg} + rising λ_2 + falling Q simultaneously indicates homogenization. These joint signatures are logged automatically in the EntropyAlerts XLSX sheet.

Sample entropy H_{samp} and the TARP deviation score form a temporal complexity diagnostic pair: high H_{samp} with high TARP confirms operation at the edge of chaos; low H_{samp} with low TARP confirms convergence; high H_{samp} with low TARP identifies noise-dominated output with no genuine attractor structure.

15. Recurrence Quantification Analysis

Module	Function	Key Inputs	Key Outputs
15 Recurrence Quantification Analysis	Identifies when and how often the system revisits the same regions of state space; LAM spikes serve as a reliable early-warning signal for phase transitions.	Lag-embedded aggregate activation vector; recurrence threshold ϵ ; minimum line length $l_{\min}$; pyrqa or NumPy	Recurrence plot $R_{\{ij\}}$; DET (Determinism); LAM (Laminarity); dashboard: LAM as primary metric

Tradition: Dynamical Systems

Recurrence Quantification Analysis (RQA) characterizes the state-space behavior of the simulation by identifying when and how often the system revisits the same regions of its trajectory. A recurrence plot $R_{\{ij\}} = \Theta(\epsilon - \|x_i - x_j\|)$ marks pairs of times (i, j) at which the network state was within distance ϵ .

RQA extracts scalar statistics from this binary matrix. Determinism (DET) is the fraction of recurrence points forming diagonal lines of length $\geq l_{\min}$, indicating periodic or quasi-periodic behavior. Laminarity (LAM) is the fraction forming vertical structures, indicating intermittency and slowing dynamics. LAM spikes are a particularly reliable early-warning signal for phase transitions: the system lingers near an unstable fixed point before escaping to a new regime, producing characteristic vertical recurrence structures well before the transition is visible in structural or causal metrics.

The module is implemented via the pyrqa library or a custom NumPy implementation, applied to the lag-embedded aggregate activation vector at each analysis window. RQA complements Lyapunov and TARP: where TARP tests for any temporal structure, RQA specifically identifies the recurrence patterns that characterize intermittency and near-critical slowing.

16. Partial Transfer Entropy and Multivariate Granger Causality

Module	Function	Key Inputs	Key Outputs
16 Partial Transfer Entropy and Multivariate Granger Causality	Separates direct causal relationships from indirect influence mediated through third nodes; provides the correct causal graph for hub identification when common-driver confounding is present.	Full pairwise TE matrix from Module 1; VAR model Granger screening ($p < 0.05$); KSG PTE computation	Sparse direct-causation graph $G_{\text{causal}}(t)$; $\text{PTE}_{\text{out}}(i)$ per node; updated hub score ζ_i using PTE

Tradition: Information-Theoretic

Pairwise transfer entropy $\text{TE}(X \rightarrow Y)$ can report a spuriously high value when X and Y share a common driver Z , because X 's history correlates with Z 's history, which in turn reduces uncertainty about Y . Partial transfer entropy (PTE) corrects this by conditioning on all other node histories simultaneously: $\text{PTE}(X \rightarrow Y \mid Z_1, \dots, Z_{\{N-2\}}) = \text{TE}(X \rightarrow Y \mid \{\text{all other nodes}\})$. This quantity is zero if and only if X provides no direct causal information about Y beyond what the rest of the network already provides.

The module runs a two-stage pipeline: (1) multivariate Granger screening using a VAR model fit over the analysis window (Geweke, 1982), retaining all connections with F-statistic $p < 0.05$; (2) KSG-based PTE computation only for the surviving connections. This reduces the $O(N^2)$ full PTE computation to a tractable subproblem for networks with N up to several hundred nodes.

The output is a sparse direct-causation graph $G_{\text{causal}}(t)$ that is reported alongside the pairwise TE matrix in the dashboard. When this module is enabled, the hub score ζ_i uses $\text{PTE}_{\text{out}}(i)$ rather than $\text{TE}_{\text{out}}(i)$, separating true causal hubs from nodes that merely correlate with shared upstream drivers.

17. Information Bottleneck Analysis (Inter-Community)

Module	Function	Key Inputs	Key Outputs
17 Information Bottleneck Analysis (Inter-Community)	Measures how much information actually passes between detected communities versus remaining within them; distinguishes structural connectivity from informational coupling.	Community assignments from Module 9; inter-community boundary edge activations; variational IB algorithm	C×C inter-community IB efficiency matrix; mean IB efficiency (dashboard); bottleneck severity flags

Tradition: Information-Theoretic

Community detection (Module 9) identifies which communities exist, but does not measure how much information actually passes between communities versus remaining within them. The Information Bottleneck module fills this gap by directly quantifying how informationally isolated each community boundary is.

The Information Bottleneck method (Tishby et al., 1999) frames information transmission as a compression-prediction trade-off: how much can a source variable X be compressed into a bottleneck variable T while retaining maximum predictive power about a target variable Y ? For each directed community pair (C_α , C_β), the module defines: X = the activation state of C_α nodes (source), T = the activation state of the inter-community boundary edges (bottleneck), Y = the activation state of C_β nodes (target).

The ratio $I(T;Y)/I(X;T)$ is the inter-community information transmission efficiency for the $C_\alpha \rightarrow C_\beta$ directed pair. Values near 1 indicate high transmission (communities are informationally coupled); values near 0 indicate high bottleneck compression (communities are informationally isolated). This quantity is reported as a C×C matrix at each analysis window.

The Fiedler value (Module 8) measures structural connectivity between communities; the IB transmission efficiency measures informational coupling. These can diverge: two communities connected by a structurally intact bridge may have low information transmission if that bridge carries only noise. The joint interpretation of λ_2 and IB efficiency is a richer diagnostic than either alone.

Plugin Performance Expansion

Several plug-ins are available that tailor this analytic framework to specific problems. In addition, users can create their own custom plugins to increase the capability of this framework.

A plugin is a single Python file that extends the Reed Framework with a new analytical module, update rule, or diagnostic. The framework discovers and loads plugins automatically — no framework source code is modified.

The Plugin Interface Standard

Every plugin is a Python class that inherits from `NxtLightPlugin` and implements one or more hook methods. The framework calls these hooks at defined points in the simulation cycle.

Hook	Called When	Typical Use
<code>on_init(engine)</code>	Once at startup	Cache config values; initialize state buffers.
<code>on_step(step, graph, snapshot)</code>	Every simulation step	Lightweight per-step tracking (e.g. node counts).
<code>on_analysis_window(step, history, graph)</code>	Every <code>analysis_window</code> steps	Run analysis; return result dict.
<code>on_rewire(step, record, graph)</code>	On every rewiring event	React to structural changes.
<code>on_complete(engine)</code>	When simulation ends	Write any final summaries.
<code>get_hints()</code>	GUI hints panel refreshes	Return list of interpretation strings.

Key Rules for Plugin Authors

1. The name attribute must exactly match the string in `simulation.toml`. Capitalization and underscores matter.
2. Return a result dict from `on_analysis_window` even on failure — use an empty result with a descriptive hint string rather than raising an exception, which would halt the entire simulation.
3. Read all configuration from `self.config` using `.get(key, default)`. Never hardcode values a user might want to change.
4. Do not import from `nxtlight` internals. Only import from `plugin_base`, the standard library, and third-party packages.
5. Scalar values in `scalar_results` must be Python floats or ints, not numpy scalars. Use `float(x)` or `int(x)` before returning.
6. If your plugin maintains state between windows, initialize it in `on_init`, not in the class body.

References

- [1] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.
- [2] Schreiber, T. (2000). Measuring information transfer. *Physical Review Letters*, 85(2), 461–464.
- [3] Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6), 066138.
- [4] Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *PNAS*, 113(15), 3932–3937.
- [5] de Silva, B. M., et al. (2020). PySINDy: A Python package for the sparse identification of nonlinear dynamics from data. *Journal of Open Source Software*, 5(49), 2104.
- [6] Coifman, R. R., & Lafon, S. (2006). Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1), 5–30.
- [7] Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2002). Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4), 511–533.
- [8] Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. *Physica D*, 65(1–2), 117–134.
- [9] Lemos, P., Jeffrey, N., Cranmer, M., Ho, S., & Battaglia, P. (2023). Sampling-based accuracy testing of posteriors from custom-built neural network samplers. *arXiv:2210.11357*.
- [10] Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1), 35–41.
- [11] Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
- [12] Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of small-world networks. *Nature*, 393(6684), 440–442.
- [13] Boccaletti, S., et al. (2006). Complex networks: Structure and dynamics. *Physics Reports*, 424(4–5), 175–308.
- [14] Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.
- [15] Takens, F. (1981). Detecting strange attractors in turbulence. In *Dynamical Systems and Turbulence*, 366–381. Springer.
- [16] Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.
- [17] Fiedler, M. (1973). Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2), 298–305.
- [18] Von Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and Computing*, 17(4), 395–416.
- [19] Girvan, M., & Newman, M. E. J. (2002). Community structure in social and biological networks. *PNAS*, 99(12), 7821–7826.
- [20] Newman, M. E. J. (2006). Modularity and community structure in networks. *PNAS*, 103(23), 8577–8582.
- [21] Blondel, V. D., et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008.
- [22] Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden. *Scientific Reports*, 9, 5233.
- [23] Liu, Y.-Y., Slotine, J.-J., & Barabási, A.-L. (2011). Controllability of complex networks. *Nature*, 473(7346), 167–173.
- [24] Yuan, Z., et al. (2013). Exact controllability of complex networks. *Nature Communications*, 4, 2447.
- [25] Pasqualetti, F., Zampieri, S., & Bullo, F. (2014). Controllability metrics for complex networks. *IEEE Trans. Control of Network Systems*, 1(1), 40–52.
- [26] Kalman, R. E. (1963). Mathematical description of linear dynamical systems. *SIAM J. Control*, 1(2), 152–192.
- [27] Hermann, R., & Krener, A. J. (1977). Nonlinear controllability and observability. *IEEE Trans. Automatic Control*, 22(5), 728–740.
- [28] Mezić, I. (2005). Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3), 309–325.
- [29] Williams, M. O., Kevrekidis, I. G., & Rowley, C. W. (2015). A data-driven approximation of the Koopman operator. *Journal of Nonlinear Science*, 25(6), 1307–1346.
- [30] Hebb, D. O. (1949). *The Organization of Behavior*. Wiley.

- [31] Jaeger, H. (2001). The echo state approach to analysing and training recurrent neural networks. GMD Technical Report 148.
- [32] Marwan, N., Romano, M. C., Thiel, M., & Kurths, J. (2007). Recurrence plots for the analysis of complex systems. *Physics Reports*, 438(5–6), 237–329.
- [33] Tishby, N., Pereira, F. C., & Bialek, W. (1999). The information bottleneck method. *Proceedings of the 37th Annual Allerton Conference*, 368–377.
- [34] Geweke, J. (1982). Measurement of linear dependence and feedback between multiple time series. *Journal of the American Statistical Association*, 77(378), 304–313.
- [35] Prigogine, I., & Stengers, I. (1984). *Order Out of Chaos*. Bantam Books.
- [36] Seifert, U. (2012). Stochastic thermodynamics, fluctuation theorems and molecular machines. *Reports on Progress in Physics*, 75(12), 126001.
- [37] Richman, J. S., & Moorman, J. R. (2000). Physiological time-series analysis using approximate entropy and sample entropy. *AJP Heart*, 278(6), H2039–H2049.
- [38] Holme, P., & Saramäki, J. (2012). Temporal networks. *Physics Reports*, 519(3), 97–125.
- [39] Massey, J. L. (1990). Causality, feedback and directed information. *Proc. ISITA*, 303–305.
- [40] Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 79(8), 2554–2558.